Multithreading

Khoa CNTT – Trường ĐHCN - ĐHQGHN

Motivating example

- Suppose we are building a music application:
  - Play music
  - Download the next song from the internet
- User expectations:
  - Music should keep playing
  - The next song should download in the background

→ How can the app perform both tasks at the same time?

Example of single-thread

```java
public class SingleThreadDemo {
    public static void main(String[] args) {
        playMusic();
        downloadSong();
    }

    public static void playMusic() {
        //Playing music
    }

    public static void downloadSong() {
       //Downloading song
    }
}
```

```
Playing music... second 1
Playing music... second 2
Playing music... second 3
Playing music... second 4
Playing music... second 5
Music finished.
Downloading song... 20%
Downloading song... 40%
Downloading song... 60%
Downloading song... 80%
Downloading song... 100%
Download completed.
```

The program finishes all the music task first, then starts the download task.

Example of multithreads

Multithreading allows a program to handle multiple tasks more efficiently

- One thread plays music
- One thread downloads data

```
Playing music... second 1
Downloading song... 20%
Downloading song... 40%
Playing music... second 2
Downloading song... 60%
Playing music... second 3
Downloading song... 80%
Playing music... second 4
Downloading song... 100%
Download completed.
Playing music... second 5
Music finished.
```

Example of multithreads

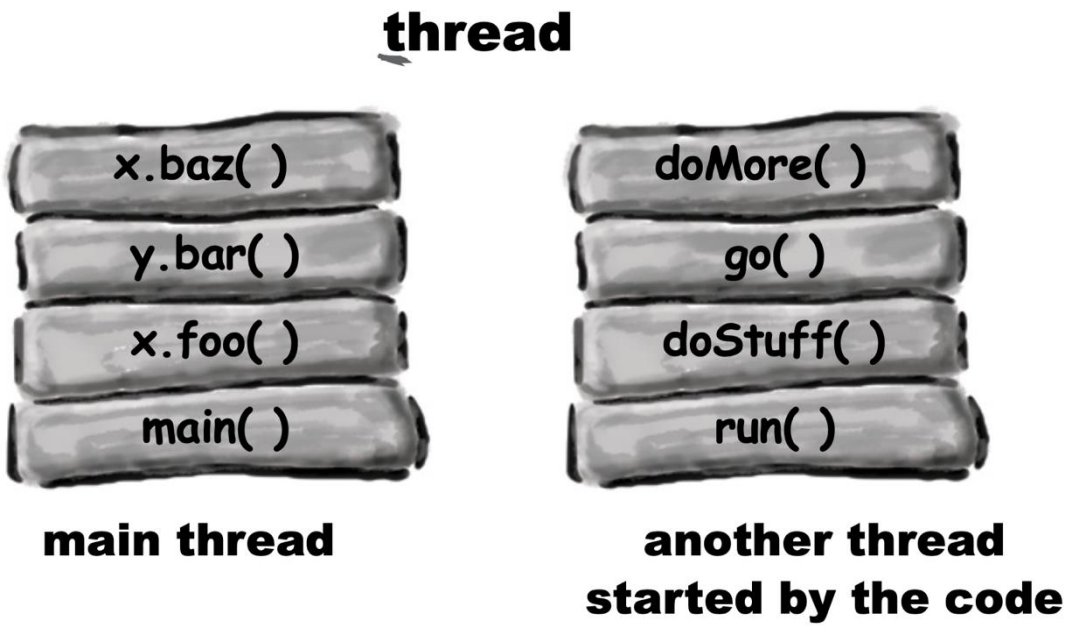
```
1 Runnable musicJob = new MusicPlayer();
2 Thread musicThread = new Thread(musicJob);
3
4 Runnable downloadJob = new SongDownloader();
5 Thread downloadThread = new Thread(downloadJob);
6
7 musicThread.start();
8 downloadThread.start();
```

What is a thread?

- Thread: a separate path of execution (a separate call stack)

In a Java program:

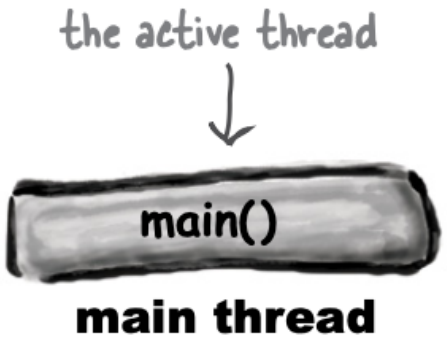
- The program starts with a main thread
- We can create additional threads to perfom other tasks by creating an instance of class Thread



What does it mean to have more than one call stack?

1 **The JVM calls the main() method.**

```
public static void main(String[] args) {
...
}
```

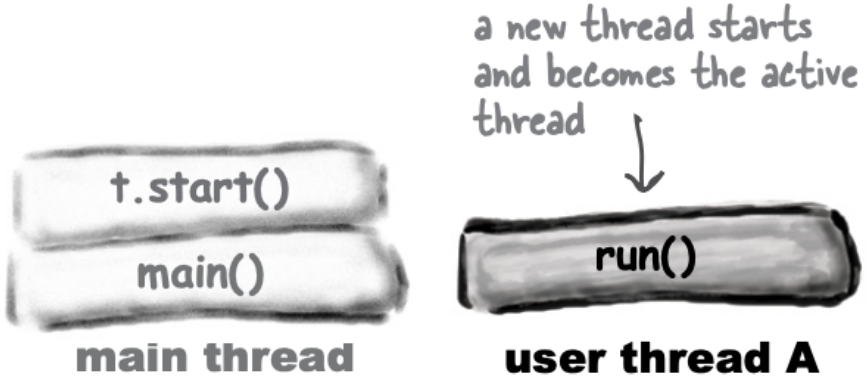


What does it mean to have more than one call stack?

2 main() starts a new thread. The main thread is temporarily frozen while the new thread starts running.

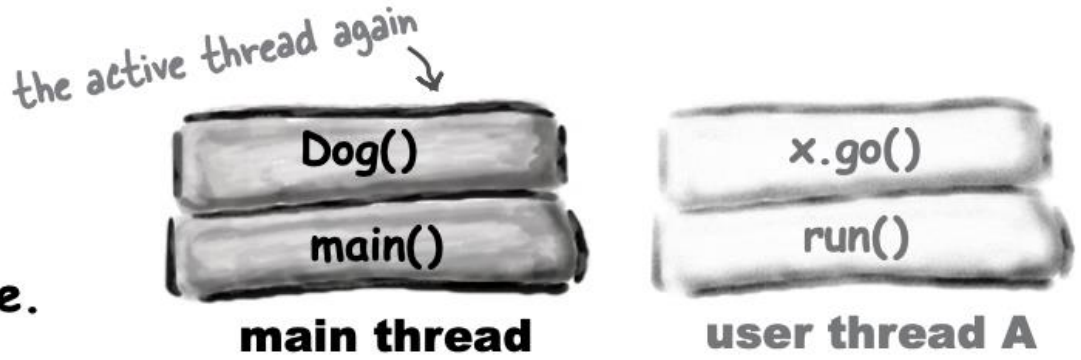
```
Runnable r = new MyThreadJob();
Thread t = new Thread(r);
t.start();
Dog d = new Dog();
```

you'll learn what this means in just a moment...



What does it mean to have more than one call stack?

3 The JVM switches between the new thread (user thread A) and the original main thread, until both threads complete.



Process vs. Thread

- Process
  - a running program (e.g., music app, browser, word)
  - has its own memory space
  - Processes are independent from each other
- Thread
  - a unit of execution inside a process
  - Shares memory and resources with other threads
  - Multiple threads can exist within one process (e.g, A music app can play music, download songs, show notifications, etc.)

Concurrency vs. Parallelism

- Concurrency:
  - Multiple tasks are in progress at the same time
  - Tasks may take turns using the CPU
  - Can happen on a single-core CPU
  - Focus: Managing multiple tasks
- Parallelism:
  - Multiple tasks are executed at the same time
  - Requires multiple CPU cores
  - Focus: Executing tasks simultaneously

Creating a new thread

1. Make a runnable object (the thread's job)

```
Runnable threadJob = new MyRunnable();
```

2. Make a Thead object (the worker) and give it a Runnable (the job)

```
Thread myThread = new Thread(threadJob);
```

3. Start the Thread

```
myThread.start();
```

What is a Runnable?

- Runnable is an interface in Java
- It represents a task/job that can be executed by a thread

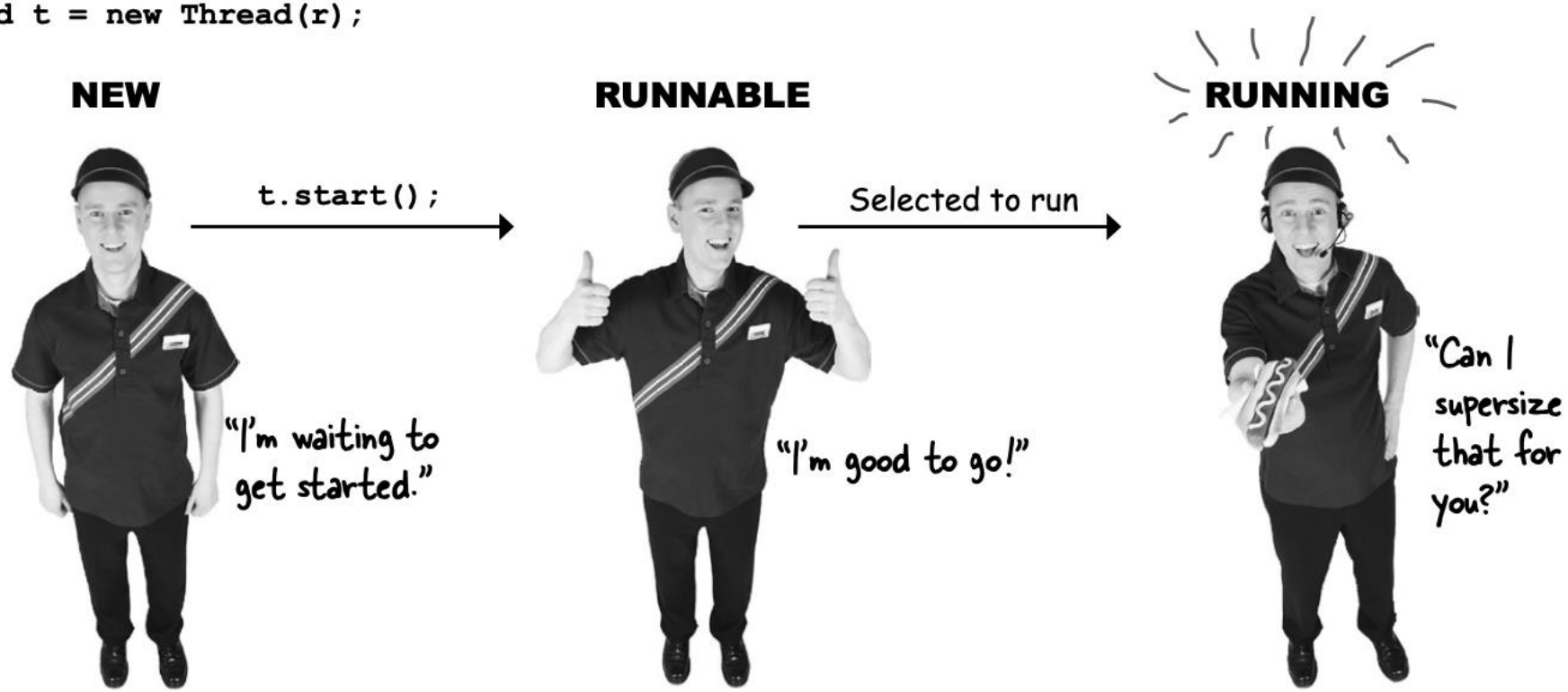
```
1 Interface Runnable{
2   void run();
3 }
```

Example of implementing Runnable

```
1  class MusicPlayer implements Runnable {
2      public void playMusic() {
3          //playing music
4      }
5
6      @Override
7      public void run() {
8          playMusic();
9      }
10 }
```

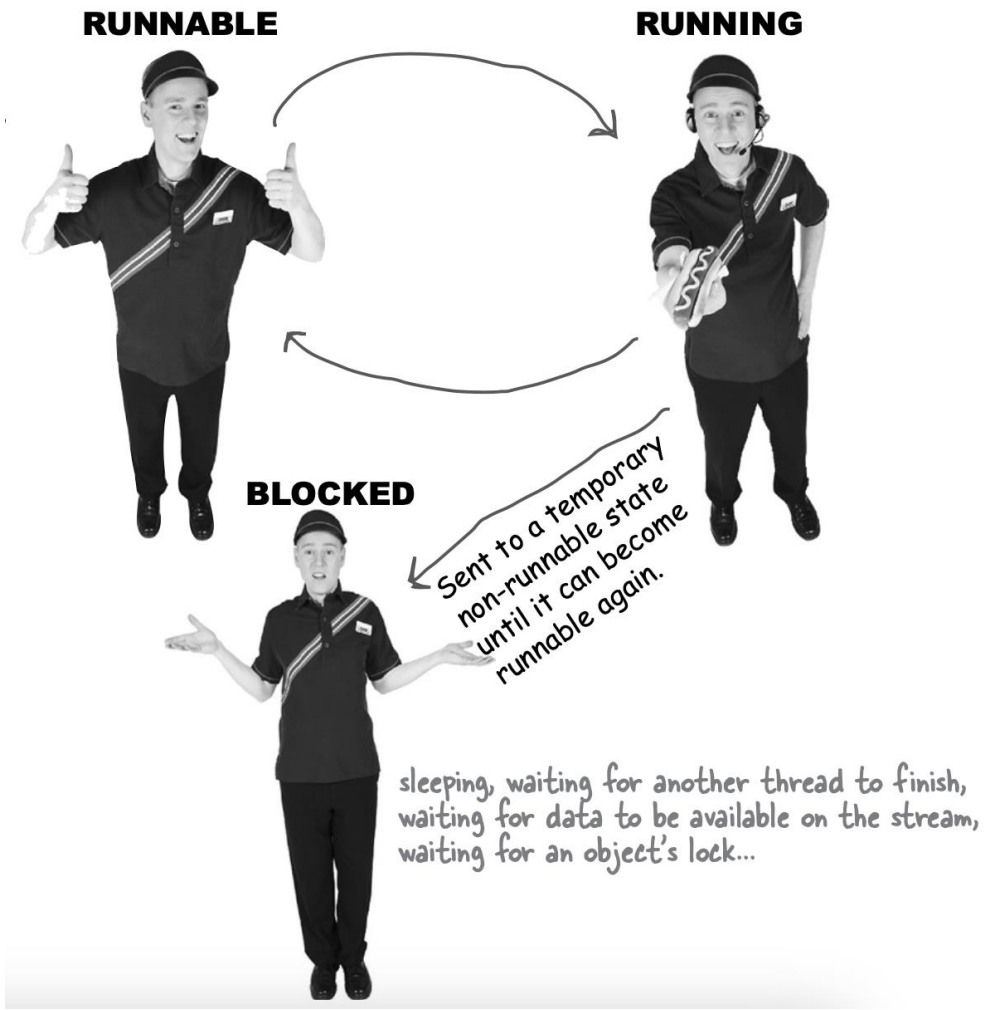
The three states of a new thread

```
Thread t = new Thread(r);
```



Typical runnable/running loop

Blocked state of a thread



Who runs next?

- When multiple threads are runnable, who decides which thread runs next?
  - The thead scheduler (part of OS and JVM)
  - It decides:
    - Which thread runs
    - When it runs
    - For how long
  - You cannot directly control the exact execution order

Scheduling strategies

- Time slicing (round-robin)
  - Each thread runs for a short time slice
  - Then scheduler switches to another thread
  - This process repeat continuously

  → Threads appear to run simultaneously
- Priority-based scheduling
  - Each thread can have a priority
  - Higher priority → more chance to run

The scheduler

- The scheduler can be unpredictable

```
Playing music... second 1
Playing music... second 2
Playing music... second 3
Playing music... second 4
Playing music... second 5
Playing music... second 6
Playing music... second 7
Playing music... second 8
Playing music... second 9
Playing music... second 10
Playing music... second 11
Playing music... second 12
Playing music... second 13
Downloading song... 5%
Playing music... second 14
Downloading song... 10%
Downloading song... 15%
Downloading song... 20%
Downloading song... 25%
Downloading song... 30%
Downloading song... 35%
Downloading song... 40%
Downloading song... 45%
```

First run

```
Downloading song... 5%
Downloading song... 10%
Downloading song... 15%
Downloading song... 20%
Downloading song... 25%
Downloading song... 30%
Downloading song... 35%
Downloading song... 40%
Downloading song... 45%
Downloading song... 50%
Downloading song... 55%
Playing music... second 1
Playing music... second 2
Playing music... second 3
Playing music... second 4
Playing music... second 5
Playing music... second 6
Playing music... second 7
Playing music... second 8
Playing music... second 9
Playing music... second 10
Playing music... second 11
Playing music... second 12
```

Second run

```
Playing music... second 1
Playing music... second 2
Playing music... second 3
Playing music... second 4
Playing music... second 5
Playing music... second 6
Playing music... second 7
Playing music... second 8
Playing music... second 9
Playing music... second 10
Playing music... second 11
Playing music... second 12
Playing music... second 13
Downloading song... 5%
Downloading song... 10%
Downloading song... 15%
Playing music... second 14
Playing music... second 15
Downloading song... 20%
Playing music... second 16
Downloading song... 25%
Playing music... second 17
Downloading song... 30%
```

Third run

Putting a thread to sleep

- Periodically putting threads to sleep allows the scheduler to switch between them more effectively

Putting a thread to sleep

- What does “sleep” mean?
  - A thread can pause its execution for a fixed time
  - During sleep:
    - The thread is not running
    - It does not consume CPU
    - It will wake up automatically after the “timer” has expired and go back to the runnable state

Example

```java
class MusicPlayer implements Runnable {
    public void playMusic() {
        for (int i = 1; i <= 20; i++) {
            System.out.println("Playing music... second " + i);
            try {
                Thread.sleep(1000);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
                System.out.println("Music player interrupted.");
            }
        }
        System.out.println("Music finished.");
    }
    @Override
    public void run() {
       playMusic();
    }
}
```

Concurrency issues

Two people sharing one bank account
- The balance is 30
- What shoud happen if both withdraw 20

→ Only one should succeed

```java
public class BankAccount {
    int balance = 30;

    public boolean withdraw(int amount) {
        if (balance >= amount) {
            balance -= amount;
            return true;
        }
        return false;
    }
```

Concurrency issues

- Balance is shared data
- Operations are not atomic
- Both can do check and withdraw at the same time → leading negative balance

```
Thread 1 trying to withdraw 20
Thread 2 trying to withdraw 20
Thread 2 completed withdrawal of 20
Balance after withdrawal: -10
Thread 1 completed withdrawal of 20
Thread 2 finished
Balance after withdrawal: -10
Thread 1 finished
```

Synchronized

- Idea: Ensure that only one thread executes withdraw at a time
- Using `synchronized`

```java
public class BankAccount {
    int balance = 30;

    public synchronized boolean withdraw(int amount) {
        if (balance >= amount) {
            balance -= amount;
            return true;
        }
        return false;
    }
```

What does synchronized do?

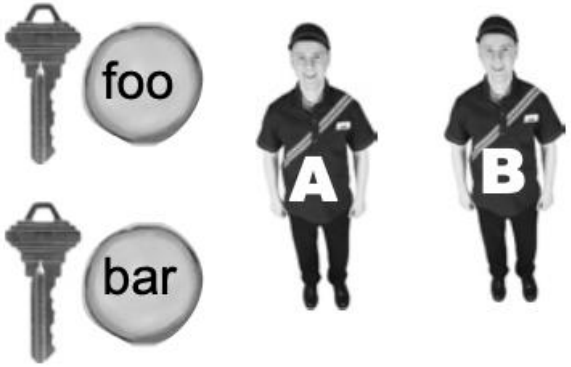
- Locks the object (e.g., a BankAccount object)
- Guarantees only one thread executes the synchronized method of the locked object
- Other threads must wait until the lock is released

Object lock

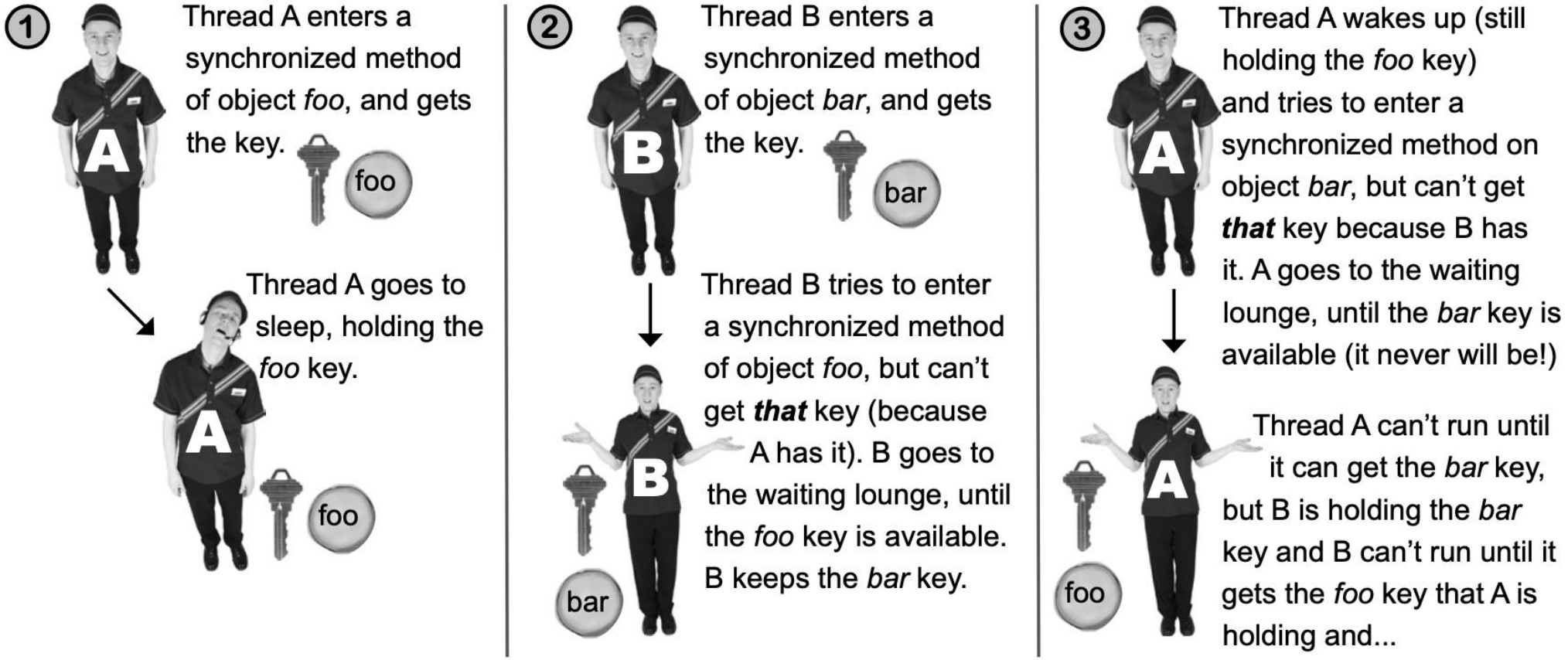
- Every Java object has an associated lock
- When a thread entors a synchronized method/block
  - It locks the objects and takes the key
  - Onle one thread can hold this key at a time
- While holding the lock
  - Other threads cannot enter any synchronized methods of the same object
  - Other non-synchronized method are still acessible

Deadlock

All it takes for deadlock are two objects and two threads.



Deadlock

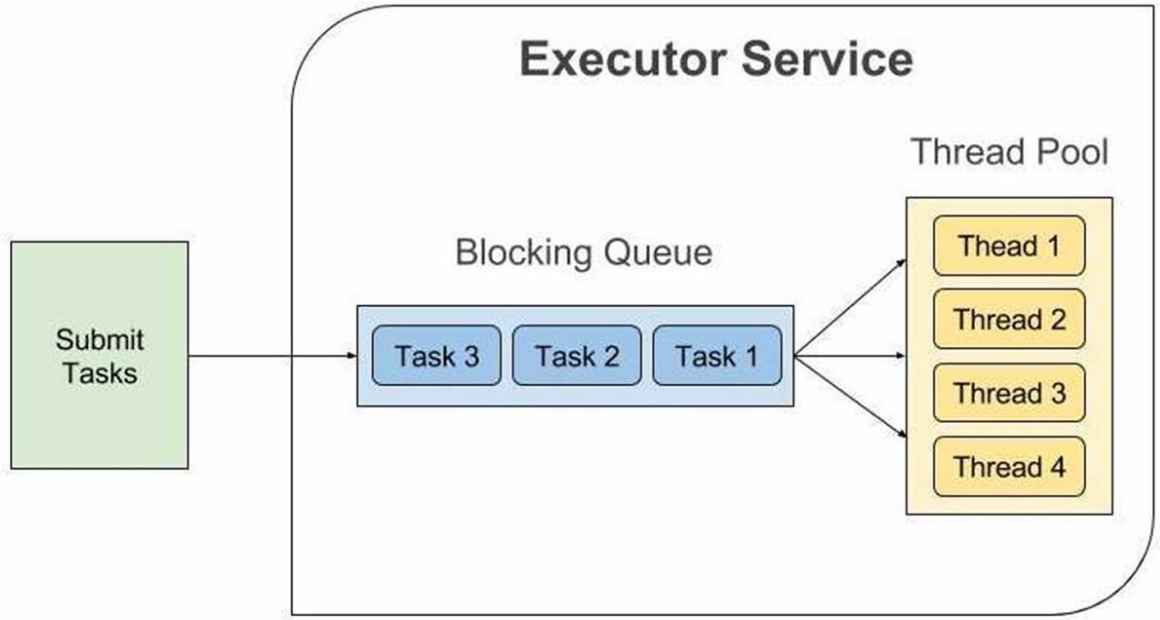

1
A
foo
Thread A enters a synchronized method of object *foo*, and gets the key.
A
foo
Thread A goes to sleep, holding the *foo* key.
2
B
bar
Thread B enters a synchronized method of object *bar*, and gets the key.
B
bar
Thread B tries to enter a synchronized method of object *foo*, but can't get ***that*** key (because A has it). B goes to the waiting lounge, until the *foo* key is available. B keeps the *bar* key.
3
A
Thread A wakes up (still holding the *foo* key) and tries to enter a synchronized method on object *bar*, but can't get ***that*** key because B has it. A goes to the waiting lounge, until the *bar* key is available (it never will be!)
A
foo
Thread A can't run until it can get the *bar* key, but B is holding the *bar* key and B can't run until it gets the *foo* key that A is holding and...

Problems with creating a Thread instance

- Using Thread directly
  - Manually create each thread
  - Could creating too many threads
  - Difficult to control execution
  - Hard to manage thread lifecycle

ExecutorService

- `ExecutorService` is a framework to manage a pool of threads and execute tasks



Creating ExecutorService

1. Create a pool of n threads

```
ExecutorService executor = Executors.newFixedThreadPool(n);
```

2. Submit tasks to the thread pool

```
executor.submit(new Task());
executor.submit(new Task());
```

3. Shutdown: stop accepting new tasks

```
executor.shutdown();
```

Example

```java
BankAccount B = new BankAccount(30);
WithdrawTask withdrawTask1 = new WithdrawTask(B, 20);
WithdrawTask withdrawTask2 = new WithdrawTask(B, 20);

ExecutorService executor = Executors.newFixedThreadPool(2);
executor.submit(withdrawTask1);
executor.submit(withdrawTask2);
executor.shutdown();
```

Limitations of Runnable

- With `Runnable`, we cannot return a result
- What if we need:
  - A computed result
  - Exception handling

```
1 class MusicPlayer implements Runnable {
2     public void run() {
3         //playing music
4     }
5 }
```

Callable

- Similar to Runnable, but
  - Returns a result
  - Can throw exceptions

```
1 public interface Callable<V>{
2   V call() throws Exception;
3 }
```

Callable example

```
1 public class Calculation implements Callable<Integer> {
2     private Integer first;
3     private Integer second;
4
5     public Calculation(Integer _first, Integer _second){
6         first = _first;
7         second = _second;
8     }
9
10    public Integer sum(){
11        return first + second;
12    }
13
14    public Integer call() throws Exception {
15        return sum();
16    }
17 }
```

Example

```
1 ExecutorService executor = Executors.newFixedThreadPool(1);
2 Calculation mytask = new Calculation(7, 8);
3 Future<Integer> result = executor.submit(mytask);
4 executor.shutdown();
5
6 System.out.println("The value of calculation:" + result.get());
```

Future

- `Future` is an interface in Java
- A `Future` represents a result that will be available later
  - A placeholder for a result that is not ready yet
- Important methods:
  - `get()`: return results, blocks until result is ready
  - `isDone()`: return true of task is finished
  - `cancel()`: attempts to cancel execution

Example

```
1 ExecutorService executor = Executors.newFixedThreadPool(1);
2 Calculation mytask = new Calculation(7, 8);
3 Future<Integer> result = executor.submit(mytask);
4 executor.shutdown();
5
6 System.out.println("The value of calculation:" + result.get());
```